WPI

RBE 550: CC-RRT FINAL PROJECT

Team-

Ethan Zhong

Harsh Chhajed

Manav Mepani

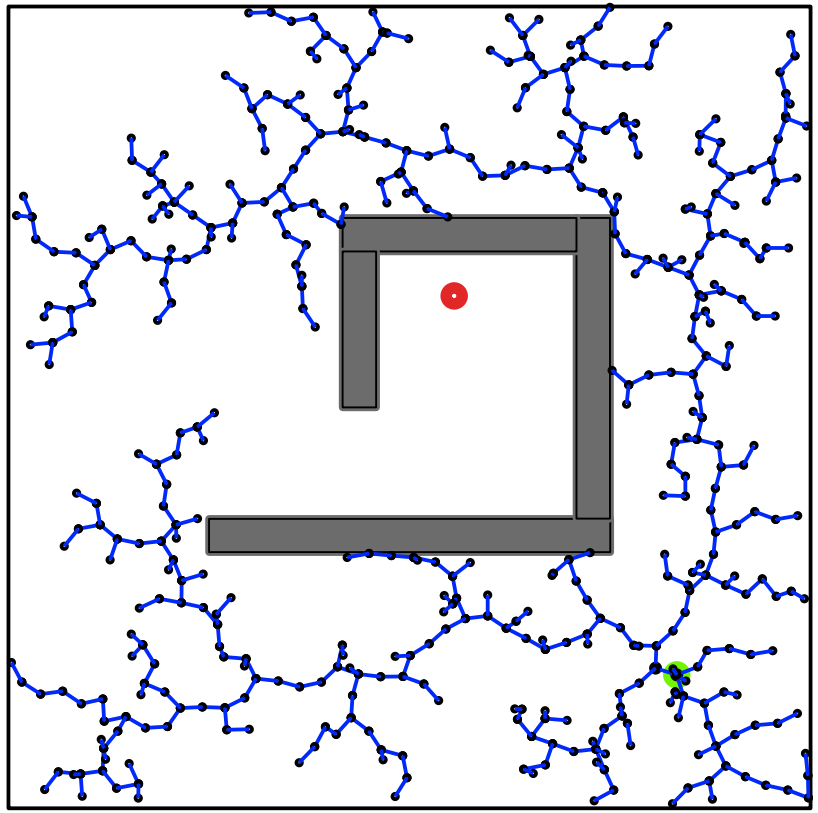
Pranay Katyal

Guided By-

Prof. Constantinos Chamzas

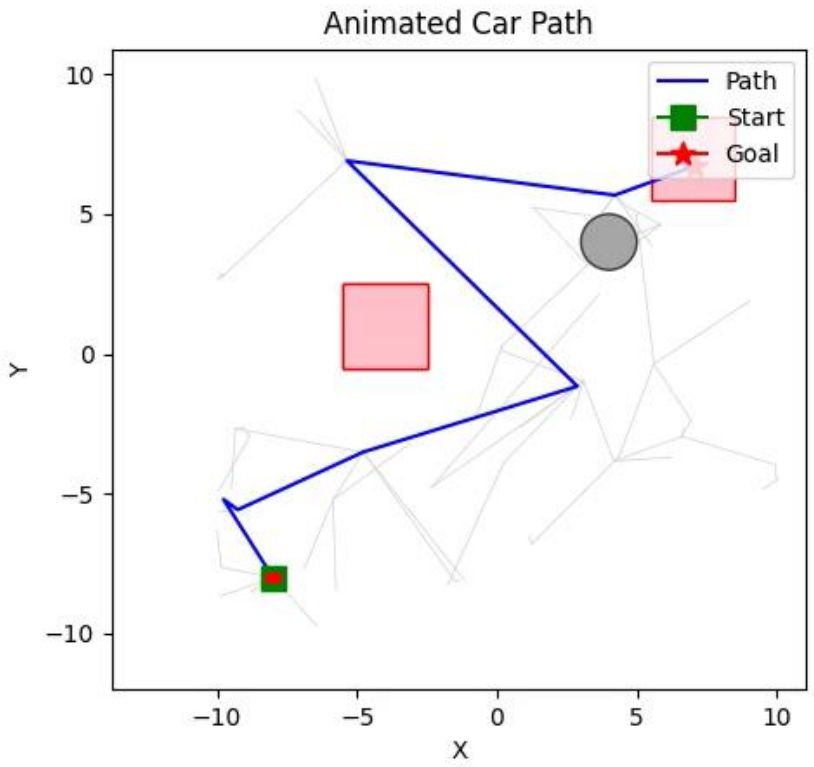
WHAT IS RRT ?

- **Rapidly-Exploring Random Trees (RRT)** is a tree-based, sampling-based motion planning algorithm used to explore high-dimensional spaces efficiently.
- It works by randomly sampling valid robot states around existing nodes and connecting each new sample to its nearest neighbor in the tree, allowing the tree to grow incrementally.



WPI

What is CC-RRT and why we need it?

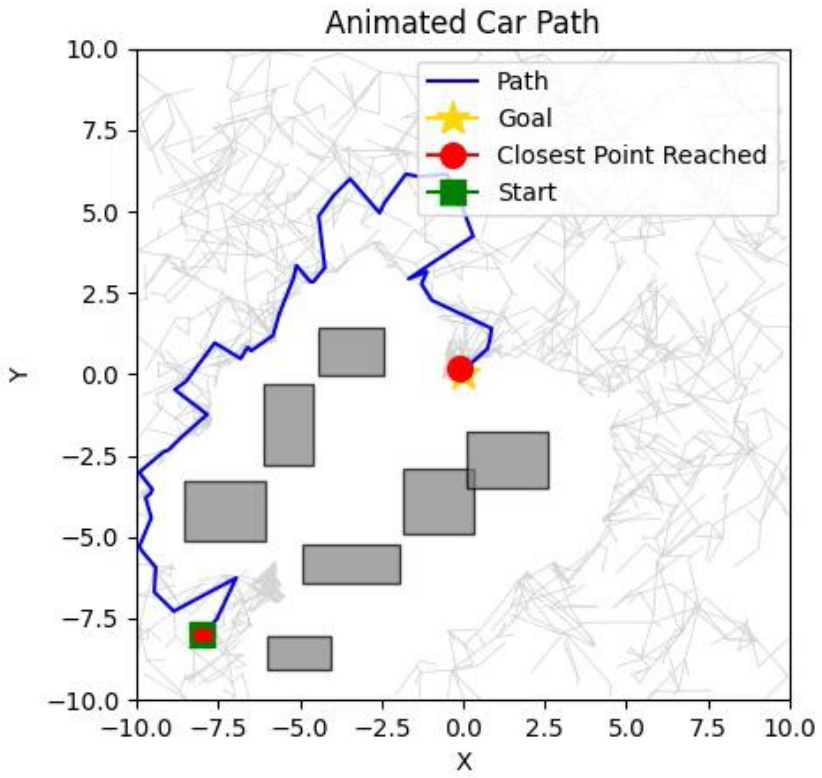


- **Chance-Constrained Rapidly-Exploring Random Trees (CC-RRT)** is an extension of RRT that incorporates uncertainty into the motion planning process, ensuring paths are probabilistically safe.
- It evaluates the likelihood of constraint violations such as collisions by modeling uncertainty (e.g., process noise, sensor errors, or moving obstacles), and ensures that the risk stays below a user-defined threshold during tree expansion.

WPI

How does CC-RRT works?

- **Uncertainty Modeling:** CCRRT extends RRT by modeling uncertainty in motion using linear dynamics and tracking both the mean state and covariance at every step.
- **Confidence Ellipses:** Each node carries a covariance ellipse, representing the region of probable obstacle positions due to process noise.
- **Chance Constraints via psafe:** CCRRT uses a safety threshold psafe to accept only those states whose probability of collision is below 1−psafe1 - psafe1−psafe.
- **Risk-Biased Expansion:** Tree growth is guided by risk-aware heuristics — nodes with lower cumulative collision risk (deltaMax) are prioritized.
- **Robustness over Optimality:** CCRRT trades path length and speed for robust, probabilistically safe trajectories — crucial for deployment in uncertain, real-world environments



WPI

WHAT IS THE DIFFERENCE BETWEEN RRT AND CC-RRT?

Aspect	RRT	CCRRT
Primary Goal	Quickly find *any* feasible path.	Find **probabilistically safe** paths under uncertainty.
Collision Checking	Checks for collisions at each sampled state and edge.	Uses **chance-constrained checks** using covariance ellipses and a psafe threshold.
Uncertainty Handling	Ignores uncertainty; assumes deterministic motion.	Propagates **mean and covariance** during planning; accounts for **process noise**.
Sampling Efficiency	May waste time near invalid or risky states.	**Biases away from high-risk regions** using delta risk and risk-based node selection.
Path Robustness	May pass close to obstacles without margin.	Enforces **safety margins** by evaluating collision probability around uncertainty bounds.
Performance in Clutter	Can degrade in tight, obstacle-dense areas.	Slower but **more reliable** in clutter, especially for high psafe values.
Tree Expansion	Greedily expands toward random samples.	Uses **risk-aware heuristics** and **branch-and-bound pruning** to avoid unsafe branches.

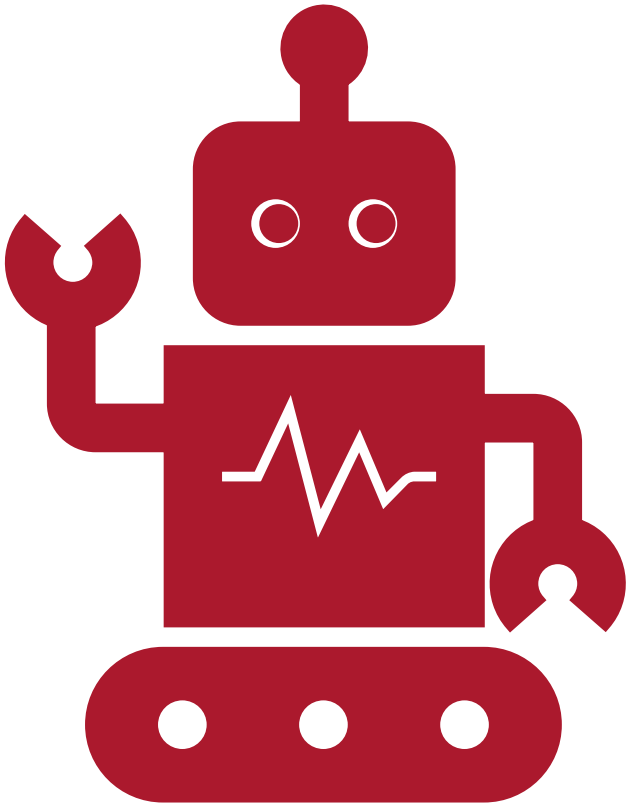
TOOLS AND PLATFORMS

Platform: C++, Python

Tools: OMPL, Docker, VS CODE, Github

WPI

Implementation Overview



- **State and Control Space Setup**: The robot's 2D position (x,y) is defined using OMPL's **RealVectorStateSpace**(2), and its 2D velocity (v_x,v_y) is represented using **RealVectorControlSpace**(2), with bounds set according to physical and environmental constraints.
- **Planner Initialization**: Obstacles, start and goal states are defined, and the CCRRT planner is configured with a specified chance constraint parameter (p-safe).
- **Uncertainty Handling**: A state propagator is set up to track both the mean and covariance of the robot's state, enabling probabilistic reasoning.
- **Validation and Execution**: The state validity checker and motion validator enforce both deterministic and chance-constrained collision avoidance, after which the planner is run and outputs the path, uncertainty, and search tree for visualization.

WPI

Implementation details – CCRRT.cpp

Uncertainty Propagation: The 'propagate' method in CCRRT.cpp not only moves the robot's mean state forward but also updates the covariance matrix, capturing how uncertainty grows as the robot moves, especially near obstacles.

Adaptive Process Noise: The process noise in the covariance update is scaled based on the robot's proximity to obstacles higher noise near obstacles, lower in free space making the planner more cautious in cluttered environments.

Risk-Biased Node Selection: The planner uses a risk-biased strategy to select which node to expand, preferring nodes with lower accumulated collision risk (**'deltaMax'**).

Branch-and-Bound Pruning: Nodes whose cost-to-go plus cost-so-far exceed the best-known solution are pruned early, improving efficiency.

Heuristic Consistency: The planner uses a consistent heuristic (Euclidean distance) to the goal for informed search and pruning.

Loop Prevention: The planner checks for loops in the tree to ensure valid tree structure.

Dynamic Feasibility: Controls are checked to ensure they do not exceed velocity bounds, enforcing realistic robot dynamics.

Output Files: The planner writes not only the solution path but also the covariance at each state and the entire search tree for post-analysis and visualization.

Implementation details – Motionvalidator.cpp

Intermediate State Sampling: The **'checkMotion'** method samples multiple points along each edge (motion) to ensure that the entire path segment, not just the endpoints, satisfies the chance constraint.

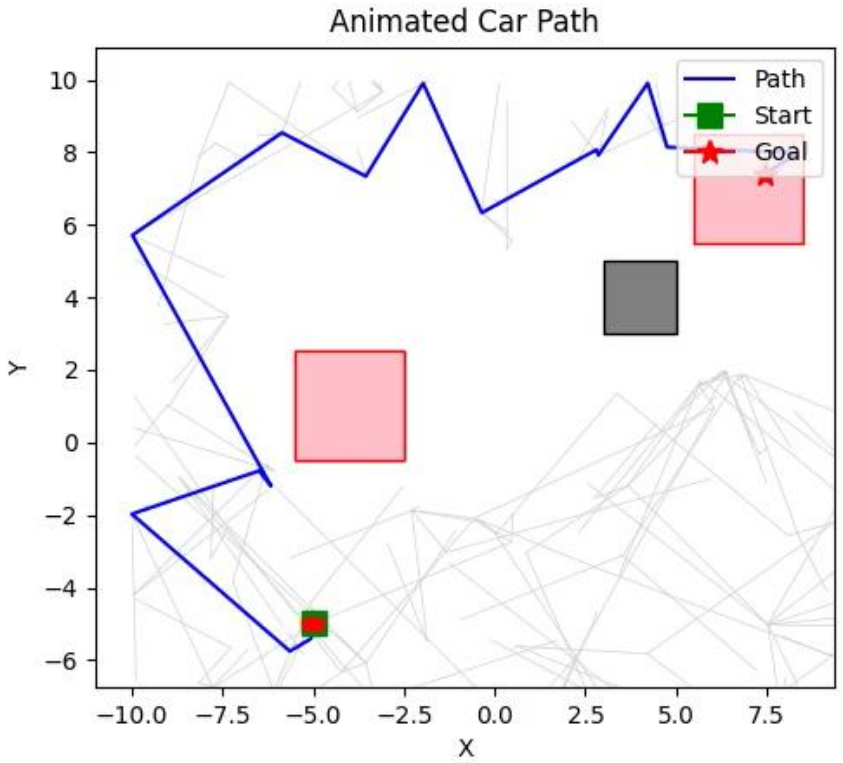
Chance Constraint Evaluation: For each sampled state, the validator uses the covariance matrix to compute the probability of collision with each obstacle, enforcing the **'p-safe'** threshold.

Obstacle Handling: Both rectangular and circular obstacles are supported, with appropriate geometric checks for each.

Integration with Uncertainty Map: The validator accesses the uncertainty map maintained by the planner to retrieve the covariance for each sampled state.

Early Termination: If any sampled state along a motion violates the chance constraint or collides with an obstacle, the motion is immediately rejected, preventing unsafe paths from being added to the tree.

Something we tried



- Tried implementing the dynamic obstacles that we know for sure are going to move and whose linear dynamics is simple and known.
- Made sure the obstacles initially covered the goal, which shows that our path is being planned with the proper algorithmic implementation.

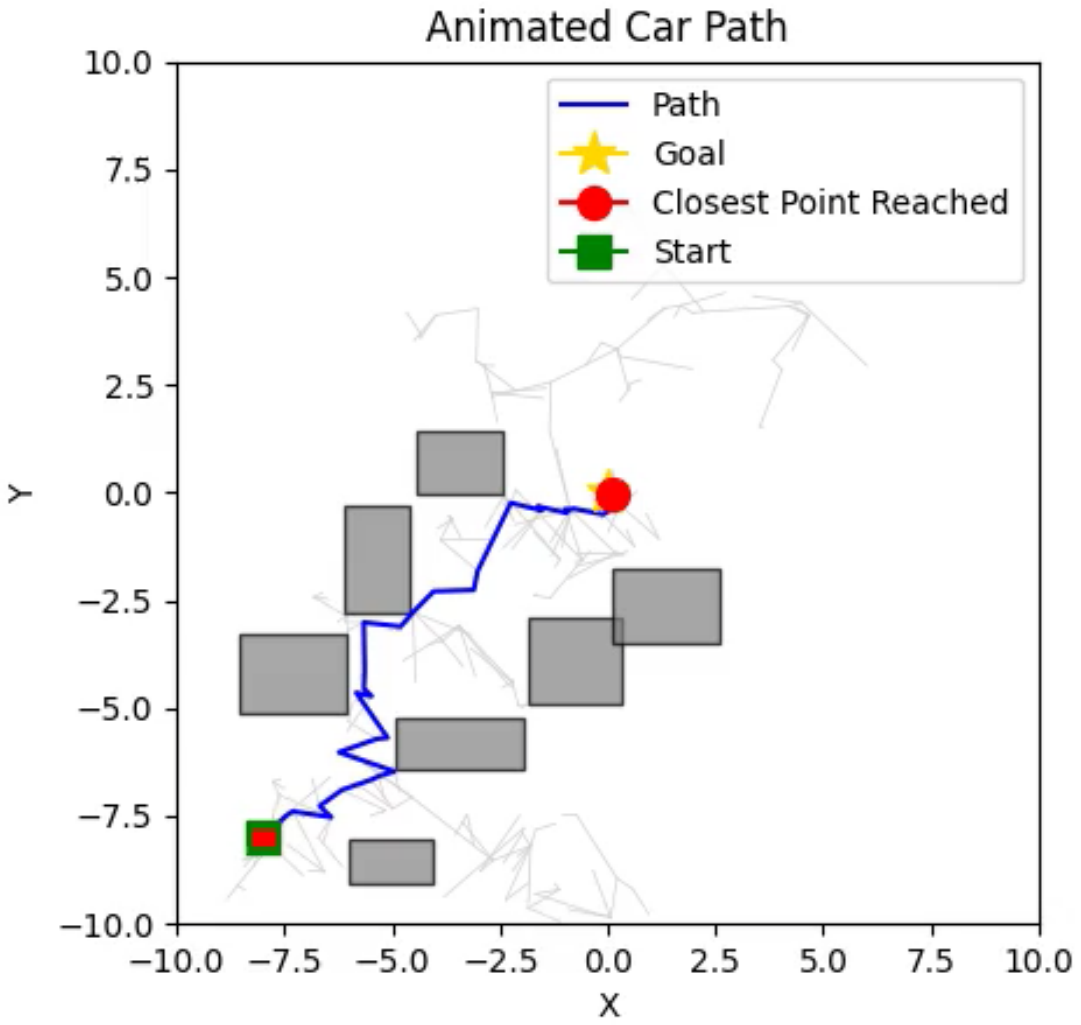
WPI

Evaluation Of Results

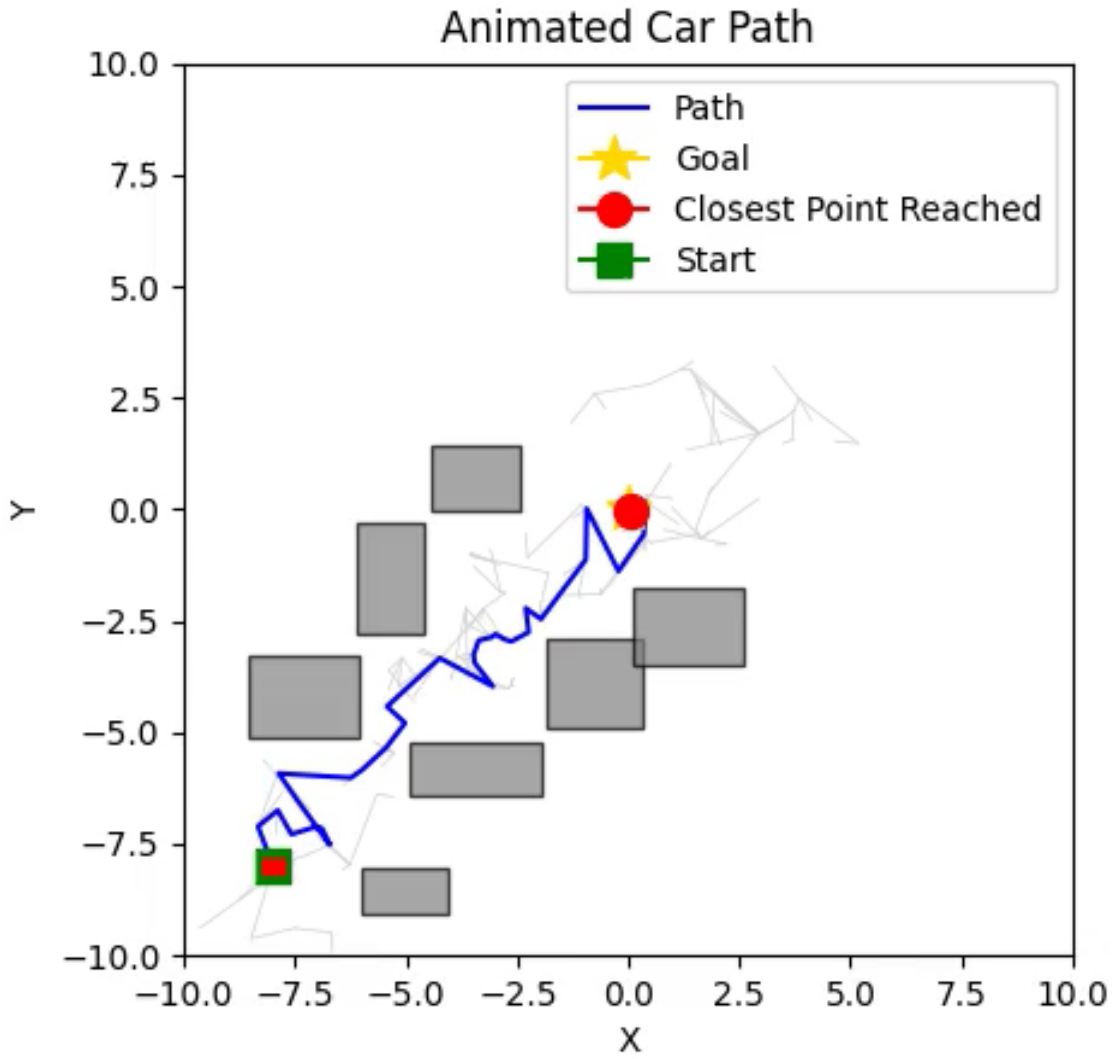
To assess the performance of the Chance-Constrained RRT (CC-RRT) algorithm, we conducted experiments using different values of the probability of safety ('p-safe'). The values tested were 0.50, 0.70, 0.90, 0.95, and 0.99. The evaluation focused on several key metrics:

1. **Quality of Path**
2. **Creation Time**
3. **Smoothness of Path**

P-safe 50

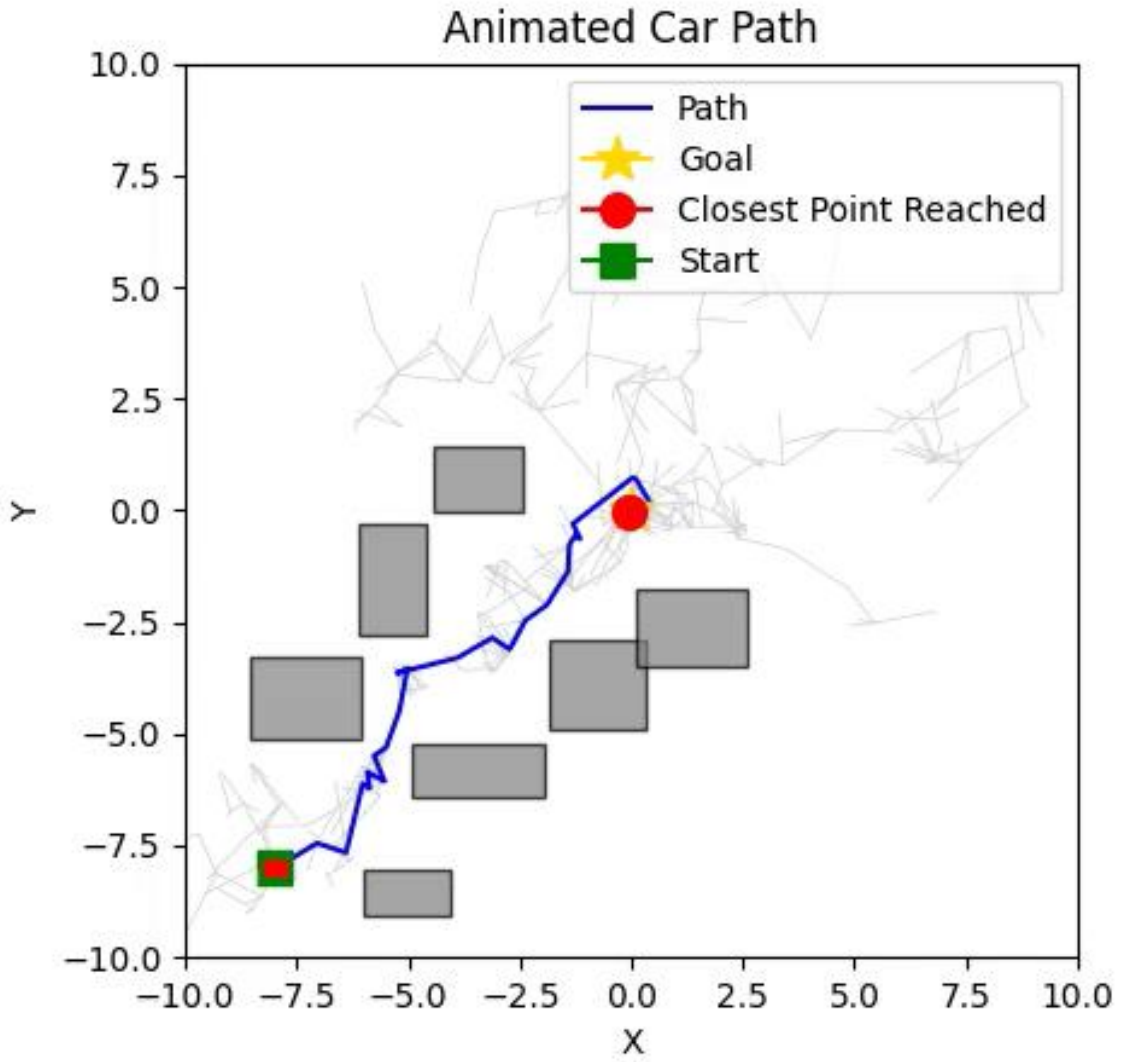

Animated Car Path
Path
Goal
Closest Point Reached
Start
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0
Y
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
X

P-safe 70


Animated Car Path
Path
Goal
Closest Point Reached
Start
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0
Y
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
X

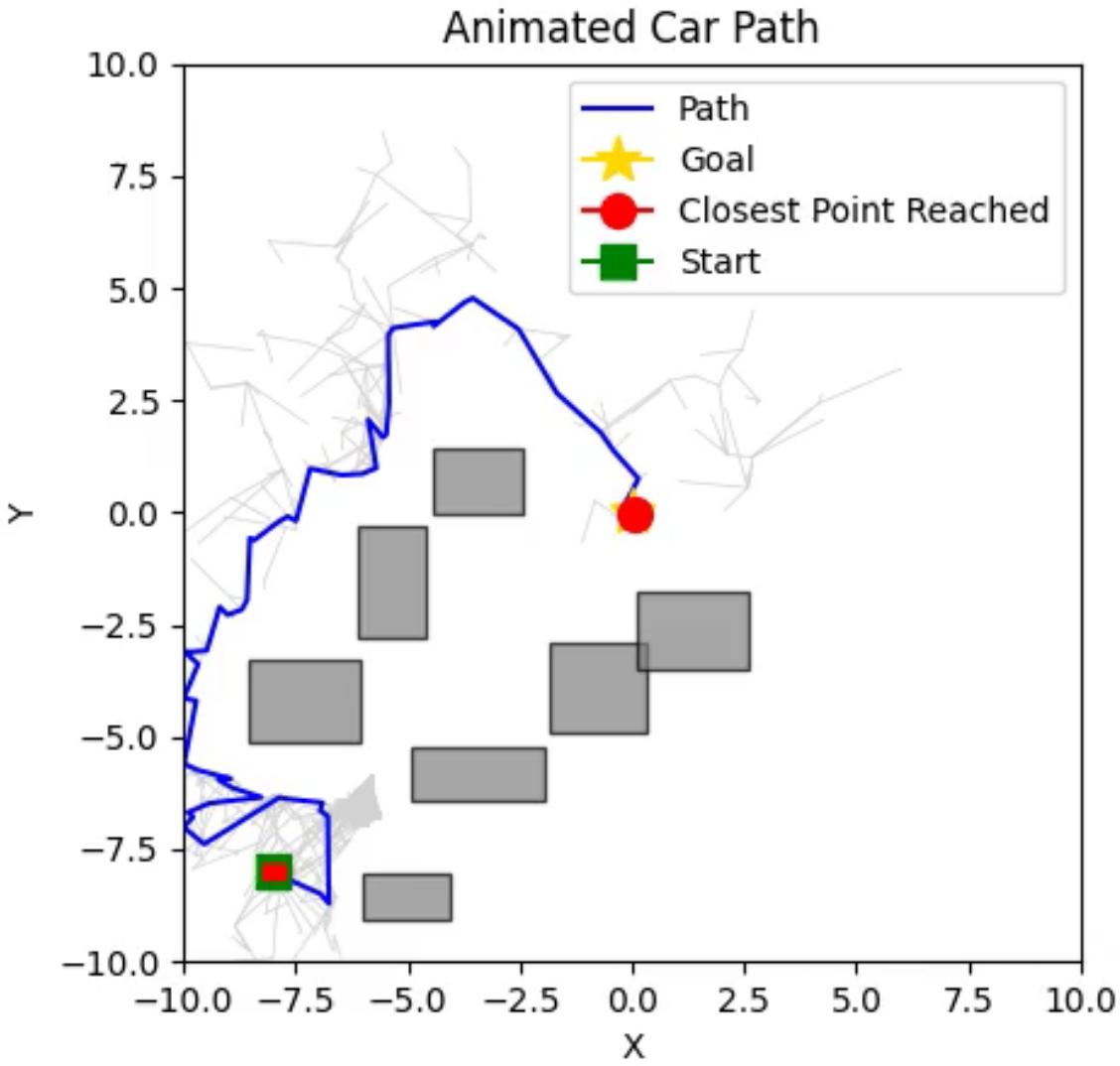
WPI

P-safe 90

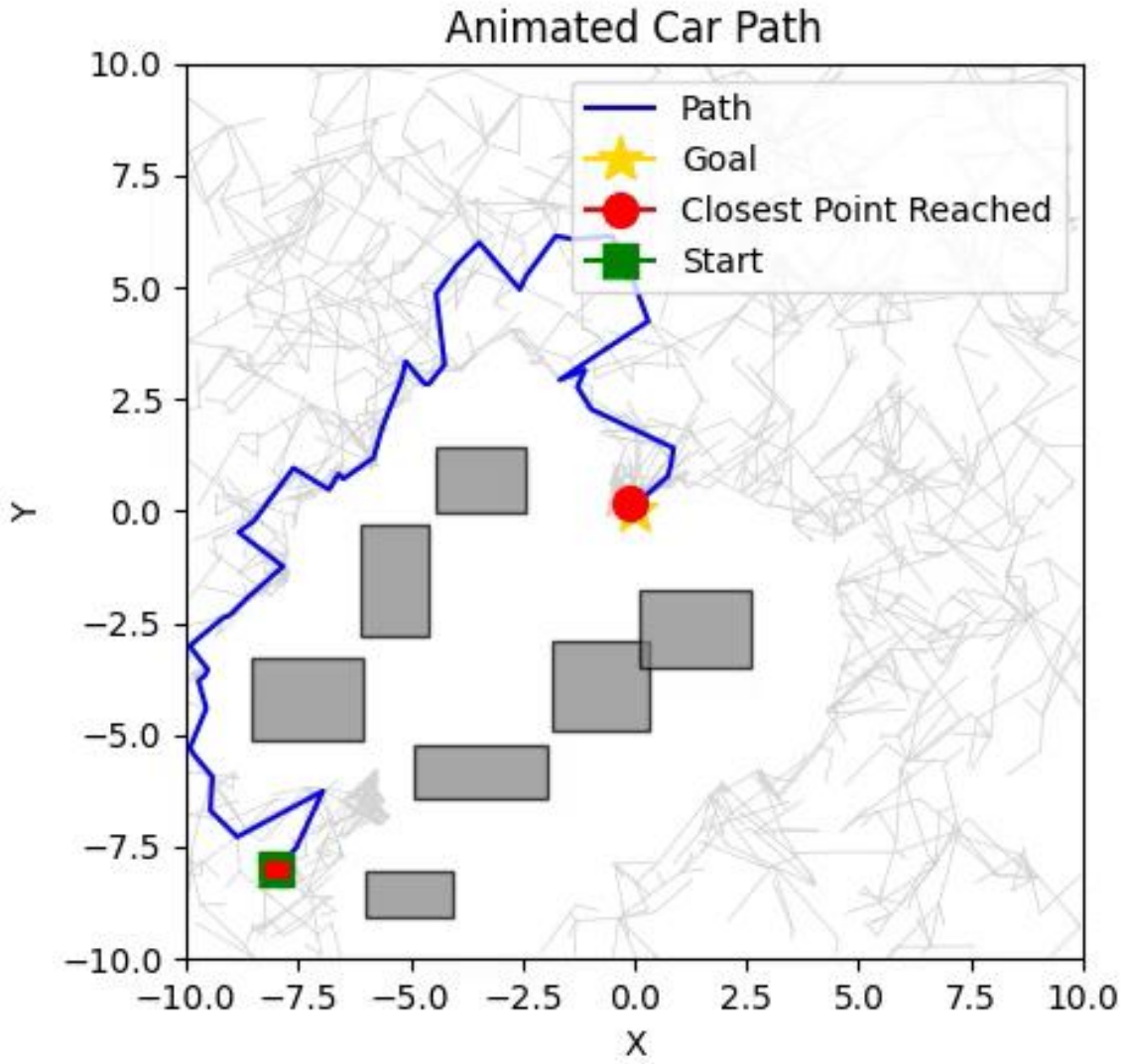

Animated Car Path
Path
Goal
Closest Point Reached
Start
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0
Y
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
X

WPI

P-safe 95


Animated Car Path
Path
Goal
Closest Point Reached
Start
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0
Y
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
X

P-safe 99


Animated Car Path
Path
Goal
Closest Point Reached
Start
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0
Y
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
X

Benchmark

PERFORMANCE IN THE CLUTTERED SCENARIO (AVERAGED OVER 10 TRIALS).

Planner	p_{safe}	Path Dur. (s)	Success Rate
CC-RRT	0.5	0.942	100%
CC-RRT	0.7	0.839	100%
CC-RRT	0.9	1.029	100%
CC-RRT	0.95	0.734	100%
CC-RRT	0.99	0.808	100%

WPI

Evaluation Of Results

1. Length/Quality of Path

- As 'p-safe' increases, the planner initially tries to go through the shorter path but eventually generates paths that maintain a greater distance from obstacles, resulting in safer but potentially longer routes
- Lower 'p-safe' values allow the planner to take more direct (riskier) paths, which may be shorter but have a higher probability of collision.

2. Creation Time

- Higher 'p-safe' values generally increase the computation time required to find a solution, as the planner must search for paths that are robust to greater uncertainty and avoid a larger safety margin around obstacles.
- For lower 'p-safe' values, the planner can find solutions more quickly due to relaxed safety requirements.

Evaluation Of Results - continued

3. Other Optimality Metrics

- **Clearance**: The minimum distance to obstacles along the path is greater for higher 'p-safe'.
- **Robustness**: Higher 'p-safe' values yield paths that are more robust to uncertainty in robot motion and sensing.
- **Success Rate**: The probability of successfully reaching the goal without collision increases with 'p-safe'.

Thank You